UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA
DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

NASKAH SOAL UJIAN TENGAH SEMESTER

Matakuliah	:	Graf Sains Data
SKS	:	3
Tanggal	:	Minggu ke-8/9, Gasal 2026/2027
Dosen	:	TIM
Materi	:	Python Driver untuk (`Neo4J`) dan Rekomendasi
Sifat	:	Individu

Petunjuk Umum

1. Ikuti langkah-langkah aktivitas individu sesuai dengan petunjuk.
2. Perhatikan poin-poin utama dalam tiap pertanyaan.

1. Tugas Pendahuluan

1.1 Persiapan dan Perkenalan Lingkungan

1. Buatlah sebuah environment baru menggunakan `venv` atau `uv` dan jalankan perintah seperti berikut

```
$ python -m venv myenv
```

atau, untuk `uv` seperti

```
$ uv venv myvenv --python 3.10
```

Pembuatan virtual environment ini sangat disarankan untuk proses isolasi development environment sehingga dapat meminimalisasi "disrupsi" paket-paket dependensi dari *local projects* di lingkungan kerja (laptop / PC) masing-masing.

2. Matikan paket virtual environment yang lain (jika ada).

```
$ conda deactivate
```

Aktifkan virtual environment baru `myvenv`

```
$ source myvenv/bin/activate
```

3. Install dependensi paket-paket seperti yang ada di dalam file requirements.txt seperti berikut

```
neo4j>=4.0,<4.3
imdbpy==2020.9.25
nose==1.3.7
cinemagoerng>=0.4
imdbinfo>=0.5
imdbpy>=2020.9.25
neo4j==4.2.1
```

dengan perintah kurang lebih sebagai berikut:

```
uv pip install -r requirements.txt
```

UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA
DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

1.2 Mempelajari Penggunaan Driver Neo4J

1. Ikuti tutorial penggunaan driver Neo4J untuk melakukan pembuatan objek dan akses objek berikut.
2. Baris berikut mendefinisikan string koneksi ke database Neo4J:

```
from neo4j import GraphDatabase, RoutingControl

URI = "neo4j://localhost:7687"
AUTH = ("neo4j", "password")
```

   Ganti informasi *credential* sesuai dengan kebutuhan.
3. Baris perintah berikut melakukan pembuatan sebuah Node berikut dengan relationship-nya.

```
def add_friend(driver, name, friend_name):
    driver.execute_query(
        "MERGE (a:Person {name: $name}) "
        "MERGE (friend:Person {name: $friend_name}) "
        "MERGE (a)-[:KNOWS]->(friend)",
        name=name, friend_name=friend_name, database_="neo4j",
    )
```

   Perhatikan perintah `execute_query`.
4. Untuk mencetak dan menjalankan keseluruhan

```
def print_friends(driver, name):
    records, _, _ = driver.execute_query(
        "MATCH (a:Person)-[:KNOWS]->(friend) WHERE a.name = $name "
        "RETURN friend.name ORDER BY friend.name",
        name=name, database_="neo4j", routing_=RoutingControl.READ,
    )
    for record in records:
        print(record["friend.name"])

with GraphDatabase.driver(URI, auth=AUTH, ) as driver:
    add_friend(driver, "Arthur", "Guinevere")
    add_friend(driver, "Arthur", "Lancelot")
    add_friend(driver, "Arthur", "Merlin")
    print_friends(driver, "Arthur")
```

Pertanyaan Pendahuluan (*Checkpint*)

1. Apa nama database yang digunakan pada tutorial ini?
2. Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!
3. Apa jenis graf pertemanan yang telah dibuat di sini?

CYPHER Total Reset

Jalankan perintah berikut untuk melakukan *truncate* pada konten database.

```
MATCH (n) DETACH DELETE n;
```

* Hati-hati dalam menggunakan perintah ini pada *shared database*.

UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA
DAN ILMU PENGETAHUAN ALAM
PRODI S1 SAINS DATA

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

1.3 Melakukan proses Impor Data dari RetailRocket ke Database Neo4J

Sumber data: *Roman Zykov*

1. Unduh data dari Kaggle dengan perintah berikut:
   (a) Windows `PowerShell`:
   ```
   curl.exe -L -o ecommerce-dataset.zip `
   ↪  https://www.kaggle.com/api/v1/datasets/download/retailrocket/ecommerce-dataset
   ```
   (b) Unix-based (Linux / MacOS):
   ```
   curl -L -o ecommerce-dataset.zip \
   ↪  https://www.kaggle.com/api/v1/datasets/download/retailrocket/ecommerce-dataset
   ```
2. Perhatikan fitur yang telah diunduh dengan rincian sebagai berikut:
   (a) `timestamp`: Waktu dalam format unix epoch.
   (b) `visitorid`: ID unik pengunjung.
   (c) `event`: Jenis interaksi *default: view, addtocart, transaction.*
   (d) `itemid`: ID unik produk.
   (e) `transactionid`: ID transaksi, hanya terisi saat event berjenis 'transaction'.
3. Buatlah mekanisme pembuatan session Neo4J baru dengan perintah kurang lebih seperti berikut (sumber: Neo4J Driver 6.4).

```python
from neo4j import GraphDatabase

driver = GraphDatabase.driver(uri, auth=(user, password))
session = driver.session(database="system")
```

atau, modifikasi dengan `argparse`, contohnya:

```python
def parse_arguments():
    parser = argparse.ArgumentParser(
        description="Impor dataset RetailRocket (events.csv) ke database Neo4J."
    )
    parser.add_argument(
        "-u", "--user", default="neo4j", help="Username Neo4j (default: neo4j)"
    )
    parser.add_argument(
        "-p",
        "--password",
        default="neo4j",
        help="Password Neo4J (default: neo4j)",
    )
    parser.add_argument(
        "-b",
        "--bolt",
        default="neo4j://127.0.0.1:7687",
        help="Neo4J URI / Bolt (default: neo4j://127.0.0.1:7687)",
    )
    parser.add_argument(
        "-s",
        "--source",
        default="data",
        help="Direktori/folder sumber data (default: data)",
    )
    return parser.parse_args()
```

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

Diawali oleh pemeriksaan keberadaan berkas `events.csv`

```
    csv_file_path = os.path.join(args.source, "events.csv")

    if not os.path.exists(csv_file_path):
        print(f"Error: File '{csv_file_path}' tidak ditemukan!")
        sys.exit(1)
```

Dilanjutkan dengan pembuatan koneksi

```
    try:
        driver = GraphDatabase.driver(
            args.bolt, auth=(args.user, args.password)
        )
        driver.verify_connectivity()
    except Exception as e:
        print(f"Gagal terhubung ke database: {e}")
        sys.exit(1)
```

Sesuaikan dengan kebutuhan Anda.

4. Anda lanjutkan dengan proses impor dengan mekanisme *batch*, kurang lebih sebagai berikut:

```
    batch_size =

    with driver.session() as session:
        create_constraints(session)

        print(f"Membaca file data dari: {csv_file_path}")
        chunks = pd.read_csv(csv_file_path, chunksize=batch_size)

        total_rows =
        total_chunks = (total_rows // batch_size) + 1

        print("Memulai proses impor data ke Neo4j...")
        for chunk in tqdm(
            chunks, total=total_chunks, desc="Mengimpor Batch Data"
        ):
            # Membersihkan data null/NaN pada transactionid
            chunk["transactionid"] = (
                chunk["transactionid"].fillna("").astype(str)
            )

            records = chunk.to_dict(orient="records")
            session.execute_write(import_batch, records)

    driver.close()
    print("Proses impor berhasil.")
```

5. Selanjutnya, Anda membutuhkan baris perintah untuk mengimpor data dalam CYPHER, contohnya:

```
def import_batch(tx, batch):
    cypher_query = """
    UNWIND $batch AS row
    MERGE (v:Visitor {visitorId: row.visitorid})
    MERGE (i:Item {itemId: row.itemid})

    WITH v, i, row
```

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

```
    CALL {
        WITH v, i, row
        WITH v, i, row WHERE row.event = 'view'
        CREATE (v)-[:VIEWED {timestamp: row.timestamp}]->(i)

        UNION ALL

        WITH v, i, row
        WITH v, i, row WHERE row.event = 'addtocart'
        CREATE (v)-[:ADDED_TO_CART {timestamp: row.timestamp}]->(i)

        UNION ALL

        WITH v, i, row
        WITH v, i, row WHERE row.event = 'transaction'
        CREATE (v)-[:BOUGHT {timestamp: row.timestamp, transactionId: row.transactionid}]->(i)
    }
    """
    tx.run(cypher_query, batch=batch)
```

2. Soal UTS

Petunjuk Khusus

1. `Deadline`: Selasa, 20/10 23:59.
2. Modifikasi kode sumber sesuai dengan kebutuhan Anda.
3. Unggah pekerjaan Anda (dokumentasi notebook `ipynb` dan penjelasan, serta dataset) pada akun Github Anda, dan tuliskan URL github Anda pada Google Classroom.
4. Sesi kelas pada jadwal masing-masing akan menanya beberapa mahasiswa secara acak.

2.1 Wrangling dan EDA (Maks. skor: 45)

1. Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yg dipilih per individu tidak boleh sama dalam 1 kelas.
   Misal: Fold training (P1) ⇒ 01/2023 - 03/2023; Fold testing (P2) 01/2024 - 06/2024.
2. Terapkan proses filter ini pada `timestamp` pada proses persiapan data, lalu simpan pada sebuah file `.csv`!
3. Lakukan query untuk mendapatkan komposisi dari *subgraph* yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!
4. Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai `User`. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!
5. Lakukan VSM terhadap simpul yang diasumsikan sebagai `Item`. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!
6. Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!
7. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan `Python`

Kampus 1 Unesa
Gedung E2 Ketintang
Surabaya 60231
datascience@unesa.ac.id
https://datascience.fmipa.unesa.ac.id

   (a) Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendiri record acuannya)!
   (b) Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil query-nya!

8. Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!

2.2 Analytics (Maks. skor: 55)

1. Pilih seorang *user* secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
   (a) tuliskan skrip `Python` dalam merekomendasikan KNN-nya!
   (b) berikan 3 item yang diiklankan untuk $k = 5$ pada penentuan KNN nya!
2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:
   (a) `Hit@5`
   (b) `NDCG@5`

— Semoga Sukses! —